

high-volume desktop computing market, which has the effect of keeping prices low due to competition, interchangeable components, and economies of scale.

Barroso and Hölzle’s recent treatise of what they dubbed “warehouse-scale computers” [18] contains a thoughtful analysis of the two approaches. The Transaction Processing Council (TPC) is a neutral, non-profit organization whose mission is to establish objective database benchmarks. Benchmark data submitted to that organization are probably the closest one can get to a fair “apples-to-apples” comparison of cost and performance for specific, well-defined relational processing applications. Based on TPC-C benchmark results from late 2007, a low-end server platform is about four times more cost efficient than a high-end shared memory platform from the same vendor. Excluding storage costs, the price/performance advantage of the low-end server increases to about a factor of twelve.

What if we take into account the fact that communication between nodes in a high-end SMP machine is orders of magnitude faster than communication between nodes in a commodity network-based cluster? Since workloads today are beyond the capability of any *single* machine (no matter how powerful), the comparison is more accurately between a smaller cluster of high-end machines and a larger cluster of low-end machines (network communication is unavoidable in both cases). Barroso and Hölzle model these two approaches under workloads that demand more or less communication, and conclude that a cluster of low-end servers approaches the performance of the equivalent cluster of high-end servers—the small performance gap is insufficient to justify the price premium of the high-end servers. For data-intensive applications, the conclusion appears to be clear: scaling “out” is superior to scaling “up”, and therefore most existing implementations of the MapReduce programming model are designed around clusters of low-end commodity servers.

Capital costs in acquiring servers is, of course, only one component of the total cost of delivering computing capacity. Operational costs are dominated by the cost of electricity to power the servers as well as other aspects of datacenter operations that are functionally related to power: power distribution, cooling, etc. [67, 18]. As a result, energy efficiency has become a key issue in building warehouse-scale computers for large-data processing. Therefore, it is important to factor in operational costs when deploying a scale-out solution based on large numbers of commodity servers.

Datacenter efficiency is typically factored into three separate components that can be independently measured and optimized [18]. The first component measures how much of a building’s incoming power is actually delivered to computing equipment, and correspondingly, how much is lost to the building’s mechanical systems (e.g., cooling, air handling) and electrical infrastructure (e.g., power distribution inefficiencies). The second component measures how much of a server’s incoming power is lost to the power supply, cooling fans, etc. The third component captures how much of the power delivered

to computing components (processor, RAM, disk, etc.) is actually used to perform useful computations.

Of the three components of datacenter efficiency, the first two are relatively straightforward to objectively quantify. Adoption of industry best-practices can help datacenter operators achieve state-of-the-art efficiency. The third component, however, is much more difficult to measure. One important issue that has been identified is the non-linearity between load and power draw. That is, a server at 10% utilization may draw slightly more than half as much power as a server at 100% utilization (which means that a lightly-loaded server is much less efficient than a heavily-loaded server). A survey of five thousand Google servers over a six-month period shows that servers operate most of the time at between 10% and 50% utilization [17], which is an energy-inefficient operating region. As a result, Barroso and Hölzle have advocated for research and development in energy-proportional machines, where energy consumption would be proportional to load, such that an idle processor would (ideally) consume no power, but yet retain the ability to power up (nearly) instantaneously in response to demand.

Although we have provided a brief overview here, datacenter efficiency is a topic that is beyond the scope of this book. For more details, consult Barroso and Hölzle [18] and Hamilton [67], who provide detailed cost models for typical modern datacenters. However, even factoring in operational costs, evidence suggests that scaling out remains more attractive than scaling up.

Assume failures are common. At warehouse scale, failures are not only inevitable, but commonplace. A simple calculation suffices to demonstrate: let us suppose that a cluster is built from reliable machines with a mean-time between failures (MTBF) of 1000 days (about three years). Even with these reliable servers, a 10,000-server cluster would still experience roughly 10 failures a day. For the sake of argument, let us suppose that a MTBF of 10,000 days (about thirty years) were achievable at realistic costs (which is unlikely). Even then, a 10,000-server cluster would still experience one failure daily. This means that any large-scale service that is distributed across a large cluster (either a user-facing application or a computing platform like MapReduce) must cope with hardware failures as an intrinsic aspect of its operation [66]. That is, a server may fail at any time, without notice. For example, in large clusters disk failures are common [123] and RAM experiences more errors than one might expect [135]. Datacenters suffer from both planned outages (e.g., system maintenance and hardware upgrades) and unexpected outages (e.g., power failure, connectivity loss, etc.).

A well-designed, fault-tolerant service must cope with failures up to a point without impacting the quality of service—failures should not result in inconsistencies or indeterminism from the user perspective. As servers go down, other cluster nodes should seamlessly step in to handle the load, and overall performance should gracefully degrade as server failures pile up. Just as important, a broken server that has been repaired

should be able to seamlessly rejoin the service without manual reconfiguration by the administrator. Mature implementations of the MapReduce programming model are able to robustly cope with failures through a number of mechanisms such as automatic task restarts on different cluster nodes.

Move processing to the data. In traditional high-performance computing (HPC) applications (e.g., for climate or nuclear simulations), it is commonplace for a supercomputer to have “processing nodes” and “storage nodes” linked together by a high-capacity interconnect. Many data-intensive workloads are not very processor-demanding, which means that the separation of compute and storage creates a bottleneck in the network. As an alternative to moving data around, it is more efficient to move the processing around. That is, MapReduce assumes an architecture where processors and storage (disk) are co-located. In such a setup, we can take advantage of data locality by running code on the processor directly attached to the block of data we need. The distributed file system is responsible for managing the data over which MapReduce operates.

Process data sequentially and avoid random access. Data-intensive processing by definition means that the relevant datasets are too large to fit in memory and must be held on disk. Seek times for random disk access are fundamentally limited by the mechanical nature of the devices: read heads can only move so fast and platters can only spin so rapidly. As a result, it is desirable to avoid random data access, and instead organize computations so that data is processed sequentially. A simple scenario¹⁰ poignantly illustrates the large performance gap between sequential operations and random seeks: assume a 1 terabyte database containing 10^{10} 100-byte records. Given reasonable assumptions about disk latency and throughput, a back-of-the-envelope calculation will show that updating 1% of the records (by accessing and then mutating each record) will take about a month on a single machine. On the other hand, if one simply reads the entire database and rewrites all the records (mutating those that need updating), the process would finish in under a work day on a single machine. Sequential data access is, literally, orders of magnitude faster than random data access.¹¹

The development of solid-state drives is unlikely to change this balance for at least two reasons. First, the cost differential between traditional magnetic disks and solid-state disks remains substantial: large-data will for the most part remain on mechanical drives, at least in the near future. Second, although solid-state disks have substantially faster seek times, order-of-magnitude differences in performance between sequential and random access still remain.

MapReduce is primarily designed for batch processing over large datasets. To the extent possible, all computations are organized into long streaming operations that

¹⁰Adapted from a post by Ted Dunning on the Hadoop mailing list.

¹¹For more detail, Jacobs [76] provides real-world benchmarks in his discussion of large-data problems.

take advantage of the aggregate bandwidth of many disks in a cluster. Many aspects of MapReduce’s design explicitly trade latency for throughput.

Hide system-level details from the application developer. According to many guides on the practice of software engineering written by experienced industry professionals, one of the key reasons why writing code is difficult is because the programmer must simultaneously keep track of many details in short term memory—ranging from the mundane (e.g., variable names) to the sophisticated (e.g., a corner case of an algorithm that requires special treatment). This imposes a high cognitive load and requires intense concentration, which leads to a number of recommendations about a programmer’s environment (e.g., quiet office, comfortable furniture, large monitors, etc.). The challenges in writing distributed software are greatly compounded—the programmer must manage details across several threads, processes, or machines. Of course, the biggest headache in distributed programming is that code runs concurrently in unpredictable orders, accessing data in unpredictable patterns. This gives rise to race conditions, deadlocks, and other well-known problems. Programmers are taught to use low-level devices such as mutexes and to apply high-level “design patterns” such as producer–consumer queues to tackle these challenges, but the truth remains: concurrent programs are notoriously difficult to reason about and even harder to debug.

MapReduce addresses the challenges of distributed programming by providing an abstraction that isolates the developer from system-level details (e.g., locking of data structures, data starvation issues in the processing pipeline, etc.). The programming model specifies simple and well-defined interfaces between a small number of components, and therefore is easy for the programmer to reason about. MapReduce maintains a separation of *what* computations are to be performed and *how* those computations are actually carried out on a cluster of machines. The first is under the control of the programmer, while the second is exclusively the responsibility of the execution framework or “runtime”. The advantage is that the execution framework only needs to be designed once and verified for correctness—thereafter, as long as the developer expresses computations in the programming model, code is guaranteed to behave as expected. The upshot is that the developer is freed from having to worry about system-level details (e.g., no more debugging race conditions and addressing lock contention) and can instead focus on algorithm or application design.

Seamless scalability. For data-intensive processing, it goes without saying that scalable algorithms are highly desirable. As an aspiration, let us sketch the behavior of an ideal algorithm. We can define scalability along at least two dimensions.¹² First, in terms of data: given twice the amount of data, the same algorithm should take at most twice as long to run, all else being equal. Second, in terms of resources: given a cluster twice

¹²See also DeWitt and Gray [50] for slightly different definitions in terms of *speedup* and *scaleup*.

the size, the same algorithm should take no more than half as long to run. Furthermore, an ideal algorithm would maintain these desirable scaling characteristics across a wide range of settings: on data ranging from gigabytes to petabytes, on clusters consisting of a few to a few thousand machines. Finally, the ideal algorithm would exhibit these desired behaviors without requiring any modifications whatsoever, not even tuning of parameters.

Other than for embarrassingly parallel problems, algorithms with the characteristics sketched above are, of course, unobtainable. One of the fundamental assertions in Fred Brook’s classic *The Mythical Man-Month* [28] is that adding programmers to a project behind schedule will only make it fall further behind. This is because complex tasks cannot be chopped into smaller pieces and allocated in a linear fashion, and is often illustrated with a cute quote: “nine women cannot have a baby in one month”. Although Brook’s observations are primarily about software engineers and the software development process, the same is also true of algorithms: increasing the degree of parallelization also increases communication costs. The algorithm designer is faced with diminishing returns, and beyond a certain point, greater efficiencies gained by parallelization are entirely offset by increased communication requirements.

Nevertheless, these fundamental limitations shouldn’t prevent us from at least striving for the unobtainable. The truth is that most current algorithms are far from the ideal. In the domain of text processing, for example, most algorithms today assume that data fits in memory on a single machine. For the most part, this is a fair assumption. But what happens when the amount of data doubles in the near future, and then doubles again shortly thereafter? Simply buying more memory is not a viable solution, as the amount of data is growing faster than the price of memory is falling. Furthermore, the price of a machine does not scale linearly with the amount of available memory beyond a certain point (once again, the scaling “up” vs. scaling “out” argument). Quite simply, algorithms that require holding intermediate data in memory on a single machine will simply break on sufficiently-large datasets—moving from a single machine to a cluster architecture requires fundamentally different algorithms (and reimplementations).

Perhaps the most exciting aspect of MapReduce is that it represents a small step toward algorithms that behave in the ideal manner discussed above. Recall that the programming model maintains a clear separation between *what* computations need to occur with *how* those computations are actually orchestrated on a cluster. As a result, a MapReduce algorithm remains fixed, and it is the responsibility of the execution framework to execute the algorithm. Amazingly, the MapReduce programming model is simple enough that it is actually possible, in many circumstances, to *approach* the ideal scaling characteristics discussed above. We introduce the idea of the “tradeable machine hour”, as a play on Brook’s classic title. If running an algorithm on a particular dataset takes 100 machine hours, then we should be able to finish in an hour on a cluster

of 100 machines, or use a cluster of 10 machines to complete the same task in ten hours.¹³ With MapReduce, this isn't so far from the truth, at least for some applications.

1.3 WHY IS THIS DIFFERENT?

“Due to the rapidly decreasing cost of processing, memory, and communication, it has appeared inevitable for at least two decades that parallel machines will eventually displace sequential ones in computationally intensive domains. This, however, has not happened.” — Leslie Valiant [148]¹⁴

For several decades, computer scientists have predicted that the dawn of the age of parallel computing was “right around the corner” and that sequential processing would soon fade into obsolescence (consider, for example, the above quote). Yet, until very recently, they have been wrong. The relentless progress of Moore’s Law for several decades has ensured that most of the world’s problems could be solved by single-processor machines, save the needs of a few (scientists simulating molecular interactions or nuclear reactions, for example). Couple that with the inherent challenges of concurrency, and the result has been that parallel processing and distributed systems have largely been confined to a small segment of the market and esoteric upper-level electives in the computer science curriculum.

However, all of that changed around the middle of the first decade of this century. The manner in which the semiconductor industry had been exploiting Moore’s Law simply ran out of opportunities for improvement: faster clocks, deeper pipelines, superscalar architectures, and other tricks of the trade reached a point of diminishing returns that did not justify continued investment. This marked the beginning of an entirely new strategy and the dawn of the multi-core era [115]. Unfortunately, this radical shift in hardware architecture was not matched at that time by corresponding advances in how software could be easily designed for these new processors (but not for lack of trying [104]). Nevertheless, parallel processing became an important issue at the forefront of everyone’s mind—it represented the only way forward.

At around the same time, we witnessed the growth of large-data problems. In the late 1990s and even during the beginning of the first decade of this century, relatively few organizations had data-intensive processing needs that required large clusters: a handful of internet companies and perhaps a few dozen large corporations. But then, everything changed. Through a combination of many different factors (falling prices of disks, rise of user-generated web content, etc.), large-data problems began popping up everywhere. Data-intensive processing needs became widespread, which drove innovations in distributed computing such as MapReduce—first by Google, and then by Yahoo

¹³Note that this idea meshes well with utility computing, where a 100-machine cluster running for one hour would cost the same as a 10-machine cluster running for ten hours.

¹⁴Guess when this was written? You may be surprised.

and the open source community. This in turn created more demand: when organizations learned about the availability of effective data analysis tools for large datasets, they began instrumenting various business processes to gather even more data—driven by the belief that more data leads to deeper insights and greater competitive advantages. Today, not only are large-data problems ubiquitous, but technological solutions for addressing them are widely accessible. Anyone can download the open source Hadoop implementation of MapReduce, pay a modest fee to rent a cluster from a utility cloud provider, and be happily processing terabytes upon terabytes of data within the week. Finally, the computer scientists are right—the age of parallel computing has begun, both in terms of multiple cores in a chip and multiple machines in a cluster (each of which often has multiple cores).

Why is MapReduce important? In practical terms, it provides a very effective tool for tackling large-data problems. But beyond that, MapReduce is important in how it has changed the way we organize computations at a massive scale. MapReduce represents the first *widely-adopted* step away from the von Neumann model that has served as the foundation of computer science over the last half plus century. Valiant called this a *bridging model* [148], a conceptual bridge between the physical implementation of a machine and the software that is to be executed on that machine. Until recently, the von Neumann model has served us well: Hardware designers focused on efficient implementations of the von Neumann model and didn't have to think much about the actual software that would run on the machines. Similarly, the software industry developed software targeted at the model without worrying about the hardware details. The result was extraordinary growth: chip designers churned out successive generations of increasingly powerful processors, and software engineers were able to develop applications in high-level languages that exploited those processors.

Today, however, the von Neumann model isn't sufficient anymore: we can't treat a multi-core processor or a large cluster as an agglomeration of many von Neumann machine instances communicating over some interconnect. Such a view places too much burden on the software developer to effectively take advantage of available computational resources—it simply is the wrong level of abstraction. MapReduce can be viewed as the first breakthrough in the quest for new abstractions that allow us to organize computations, not over individual machines, but over entire clusters. As Barroso puts it, the datacenter *is* the computer [18, 119].

To be fair, MapReduce is certainly not the first model of parallel computation that has been proposed. The most prevalent model in theoretical computer science, which dates back several decades, is the PRAM [77, 60].¹⁵ In the model, an arbitrary number of processors, sharing an unboundedly large memory, operate synchronously on a shared input to produce some output. Other models include LogP [43] and BSP [148].

¹⁵More than a theoretical model, the PRAM has been recently prototyped in hardware [153].

For reasons that are beyond the scope of this book, none of these previous models have enjoyed the success that MapReduce has in terms of adoption and in terms of impact on the daily lives of millions of users.¹⁶

MapReduce is the most successful abstraction over large-scale computational resources we have seen to date. However, as anyone who has taken an introductory computer science course knows, abstractions manage complexity by hiding details and presenting well-defined behaviors to users of those abstractions. They, inevitably, are imperfect—making certain tasks easier but others more difficult, and sometimes, impossible (in the case where the detail suppressed by the abstraction is exactly what the user cares about). This critique applies to MapReduce: it makes certain large-data problems easier, but suffers from limitations as well. This means that MapReduce is not the final word, but rather the first in a new class of programming models that will allow us to more effectively organize computations at a massive scale.

So if MapReduce is only the beginning, what's next beyond MapReduce? We're getting ahead of ourselves, as we can't meaningfully answer this question before thoroughly understanding what MapReduce can and cannot do well. This is exactly the purpose of this book: let us now begin our exploration.

1.4 WHAT THIS BOOK IS NOT

Actually, not quite yet... A final word before we get started. This book is about MapReduce algorithm design, particularly for text processing (and related) applications. Although our presentation most closely follows the Hadoop open-source implementation of MapReduce, this book is explicitly *not* about Hadoop programming. We don't for example, discuss APIs, command-line invocations for running jobs, etc. For those aspects, we refer the reader to Tom White's excellent book, "Hadoop: The Definitive Guide", published by O'Reilly [154].

¹⁶Nevertheless, it is important to understand the relationship between MapReduce and existing models so that we can bring to bear accumulated knowledge about parallel algorithms; for example, Karloff et al. [82] demonstrated that a large class of PRAM algorithms can be efficiently simulated via MapReduce.

MapReduce Basics

The only feasible approach to tackling large-data problems today is to divide and conquer, a fundamental concept in computer science that is introduced very early in typical undergraduate curricula. The basic idea is to partition a large problem into smaller sub-problems. To the extent that the sub-problems are independent [5], they can be tackled in parallel by different workers—threads in a processor core, cores in a multi-core processor, multiple processors in a machine, or many machines in a cluster. Intermediate results from each individual worker are then combined to yield the final output.¹

The general principles behind divide-and-conquer algorithms are broadly applicable to a wide range of problems in many different application domains. However, the details of their implementations are varied and complex. For example, the following are just some of the issues that need to be addressed:

- How do we break up a large problem into smaller tasks? More specifically, how do we decompose the problem so that the smaller tasks can be executed in parallel?
- How do we assign tasks to workers distributed across a potentially large number of machines (while keeping in mind that some workers are better suited to running some tasks than others, e.g., due to available resources, locality constraints, etc.)?
- How do we ensure that the workers get the data they need?
- How do we coordinate synchronization among the different workers?
- How do we share partial results from one worker that is needed by another?
- How do we accomplish all of the above in the face of software errors and hardware faults?

In traditional parallel or distributed programming environments, the developer needs to explicitly address many (and sometimes, all) of the above issues. In shared memory programming, the developer needs to explicitly coordinate access to shared data structures through synchronization primitives such as mutexes, to explicitly handle process synchronization through devices such as barriers, and to remain ever vigilant for common problems such as deadlocks and race conditions. Language extensions, like

¹We note that promising technologies such as quantum or biological computing could potentially induce a paradigm shift, but they are far from being sufficiently mature to solve real world problems.